INTERNATIONAL ISLAMIC UNIVERSITY CHITTAGONG

Department of Computer Science and Engineering

LABORATORY MANUAL

CSE-2340
SOFTWARE DEVELOPMENT 1

Comprehensive Student Laboratory Manual

3rd Semester - 2 Credit Hours
Laboratory / Practical
15 Teaching Weeks - 4 × 50-minute classes per week

Prepared in alignment with the Revised CSE-2340 Course Syllabus
Department of Computer Science and Engineering
International Islamic University Chittagong

Preface and Instructions for Students

This manual is a practical companion to the revised CSE-2340 Software Development 1 syllabus. It is organized as a laboratory manual rather than a second syllabus: each lab explains the purpose of the work, introduces the relevant concepts, provides a guided procedure, gives practical tasks, and asks you to test and explain your implementation.

The weekly sequence, assessment marks, project evaluation weeks and technical boundaries follow the supplied syllabus. Examples and practice tasks in this manual are instructional scaffolding. They do not create additional marks or replace a task brief issued by the instructor.

How to Work Through a Lab

1. Read the objectives and problem statement before opening an AI assistant or writing code.
2. Inspect the starter files or create the folder structure specified in the procedure.
3. Follow the guided steps in order. Run the application after each meaningful change.
4. Complete the independent exercises without simply copying the worked example.
5. Test normal inputs, invalid inputs, empty states and relevant screen sizes.
6. Record one issue you encountered, how you investigated it and how you verified the fix.
7. Commit your working files to GitHub and prepare to explain the implementation.

AI-Assisted Development Workflow

AI is used as a practical development assistant throughout this manual. Students practise asking focused questions, generating alternatives, reviewing code, creating test cases, debugging and improving their own implementations. AI output is a proposal-not proof that the code is correct.

1. Frame the task: state the requirement, context, existing code, expected behavior and constraints.

2. Ask deliberately: request an explanation, plan, test cases, review or debugging hint before requesting a complete solution.

3. Inspect the response: check it against the syllabus, the project setup and official documentation.

4. Implement: write or adapt the code and keep track of your own changes.

5. Verify: run the application, test normal and failure cases, inspect Developer Tools and record actual results.

6. Reflect: explain what you accepted, modified or rejected and why.

Never share passwords, API keys, private keys, personal data or confidential information with an AI assistant.

Course Information

Item	Details
Course Code / Title	CSE-2340 - Software Development 1
Semester / Credit	3rd Semester / 2 Credit Hours
Contact Hours	4 laboratory classes per week; 50 minutes each; 200 minutes total
Course Type	Core, Engineering
Prerequisite	CSE-1222 (Computer Programming 2 Lab)
Co-requisite	CSE-1221 (Computer Programming 2)
Delivery	Laboratory / Practical
Structure	15 teaching weeks × 4 classes; two 100-minute studio blocks per week; separate 100-minute mid-term practical slot

Assessment Structure

Component	Marks	Format
Attendance	10	Continuous
Mid-term In-lab Practical	20	Individual, 100-minute practical assessment
Six Mini Projects / Continuous Assessment	40	Individual projects with dedicated evaluation classes
Final Group Project / Showcase	30	Team project, individual ownership verification and viva
Total	100	

Mini-project distribution: MP1 = 5, MP2 = 5, MP3 = 7, MP4 = 7, MP5 = 8, MP6 = 8 marks. The six projects are individual and progressive.

Standard Studio Rhythm

Day 1 - Problem Discovery and Implementation	Day 2 - Engineering, Debugging and Verification
0–10: Understand the problem, constraints and expected outcome.	0–10: Review previous work and unfinished tasks.

10–20: Analyze requirements and plan with an AI assistant.	10–35: Implement additional features or improve the solution.
20–55: Implement independently; instructor asks diagnostic questions.	35–55: Debug, test, inspect code and verify AI suggestions.
55–70: Run the application and identify errors.	55–75: Respond to an unexpected requirement or change request.
70–90: Implement an extension or additional requirement.	75–90: Modify independently and explain decisions.
90–100: Commit work and submit progress record.	90–100: Individual explanation, commit and exit assessment.

Development Environment Checklist

- VS Code or another instructor-approved code editor.
- A modern web browser with Developer Tools.
- Node.js and npm for the development tasks that require them.
- Git installed and a GitHub account for repository work.
- A terminal/command prompt for project commands.
- Access to the official documentation listed in the syllabus.
- An AI assistant of your choice for regular studio activities, subject to course policy.

Do not submit passwords, API keys, private keys, personal data or confidential information to AI tools or public repositories.

Lab 1 - Web Architecture, Development Setup, Semantic HTML and Git

Lab number: 01 | Course: CSE-2340 Software Development 1 | Syllabus week: 1

Learning Objectives

- Explain the browser-based client/server model and the purpose of DNS and HTTP.
- Create and run a first HTML document using the course development environment.
- Structure a page with semantic HTML elements.
- Create a Git repository and record project changes with commits.

Concept Notes

Client and server - The browser is the client. It requests a resource from a server, which returns a response. A web page may also request images, stylesheets, scripts and data.

DNS - DNS translates a human-readable domain name into an IP address used to locate a server.

HTTP request/response - A request contains a method, URL and headers; a response contains a status code, headers and possibly a body.

Semantic HTML - Semantic elements communicate the purpose of content, not merely its visual appearance.

Git - Git records changes to files so you can inspect, restore and share project history.

Required Tools / Setup

- VS Code, a modern browser and browser Developer Tools.
- The project files and tools required for this week's topic.
- Git/GitHub for version-controlled work; use the course repository workflow.

Guided Laboratory Procedure

8. Create a folder named sd1-lab01. Open it in VS Code.
9. Create index.html. Enter the HTML document skeleton shown below and save it.
10. Open the file with Live Server or the browser. Inspect the rendered page.
11. Open Developer Tools. Inspect the Elements panel and Network panel. Reload and observe the document request.
12. Create a Git repository in the project folder. Add a .gitignore if needed, stage the files and create the first commit.
13. Create a GitHub repository and connect/push the local repository using the commands demonstrated by the instructor.

Worked Example / Reference Pattern

The following is a learning example. Adapt it to the task requirements and the project setup used in your class.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Profile</title>
</head>
<body>
  <header>
    <h1>Student Profile</h1>
    <nav aria-label="Main navigation">
      <a href="#about">About</a>
      <a href="#courses">Courses</a>
    </nav>
  </header>
  <main>
    <section id="about">
      <h2>About Me</h2>
      <p>I am learning front-end development.</p>
    </section>
    <section id="courses">
      <h2>My Courses</h2>
      <ul><li>Software Development 1</li></ul>
    </section>
  </main>
  <footer><p>Student learning project</p></footer>
</body>
</html>
```

Independent Lab Exercises

14. Add a second section describing a project you want to build.
15. Add an image with meaningful alternative text. Verify the page still communicates the image's purpose if the image fails to load.

16. Add a link to an external resource. Use an appropriate target and safe rel attributes if opening a new tab.
17. Change the document title and verify the browser tab.
18. Make at least two meaningful commits: initial page and semantic improvements.

Verification Checklist

- The page opens without a blank screen or malformed structure.
- The document has one main heading and a logical heading hierarchy.
- Navigation links move to the correct sections.
- The image has an alt attribute appropriate to its purpose.
- The repository contains the HTML file and a useful commit history.

Troubleshooting Notes

- If Live Server is unavailable, open the HTML file directly in a browser and check the path.
- If Git says a command is unknown, verify Git installation and restart the terminal.
- If push is rejected, inspect the remote URL and repository history before force-pushing; do not overwrite work blindly.

Lab Record and Reflection

Record item	Student notes
What I implemented	
One issue or error I encountered	
How I investigated and fixed it	
How I verified the result	
What I can now explain or modify independently	

AI-ASSISTED LAB ACTIVITY

AI Activity 1 - Ask AI to Explain the Web and Build a Semantic Page

Copyable prompt - replace bracketed text with your own work:

> I am a CSE-2340 student learning web fundamentals. Explain the browser client-server request/response process using a simple example. Then help me plan a semantic HTML student profile page containing a header, navigation, main content, two sections and a footer. Do not give me the complete code immediately. First give me a short structure, explain why each semantic element is appropriate, and then ask me to implement it.

Procedure

1. Ask the assistant to explain DNS, an HTTP request and an HTTP response in the context of opening a web page.
2. Use the prompt to obtain a page outline; implement the HTML yourself before requesting code.
3. Ask AI to review your HTML for semantic structure, heading order, missing attributes and invalid nesting.
4. Introduce one deliberate mistake (for example, a broken navigation target) and ask AI for a debugging hint, not the final answer.
5. Use Git to commit your original implementation and the corrected implementation separately.

Required evidence / submission

• Submit the exact prompt and a short record of the response you used.

• Submit your HTML file and Git commit history.

• Explain one suggestion you accepted and one suggestion you independently checked or rejected.

Verification rule: run the code, test the behavior and check important claims against browser evidence or official documentation. You remain responsible for understanding and modifying the submitted work.

Lab 2 - HTML Navigation, Forms, Tables, Accessibility and CSS Fundamentals

Lab number: 02 | Course: CSE-2340 Software Development 1 | Syllabus week: 2

Learning Objectives

- Build navigation, forms and tables using appropriate HTML elements.
- Apply basic accessibility practices to page structure and controls.
- Explain selectors, CSS application methods and the box model.
- Style and debug a basic page using browser Developer Tools.

Concept Notes

Forms - A form groups controls that collect user input. Each control should have an understandable label and an appropriate input type.

Tables - Tables represent tabular data. Use header cells to identify the meaning of columns.

Accessibility - Accessible structure supports keyboard use, assistive technologies and clearer interaction.

CSS cascade - When multiple rules apply, the browser resolves them using origin, importance, specificity and source order.

Box model - An element's rendered size is affected by content, padding, border and margin.

Required Tools / Setup

- VS Code, a modern browser and browser Developer Tools.
- The project files and tools required for this week's topic.
- Git/GitHub for version-controlled work; use the course repository workflow.

Guided Laboratory Procedure

19. Create a form with name, email and course fields. Associate each visible label with its input.
20. Create a table showing at least three courses and their credit hours. Use thead and tbody.
21. Add a navigation menu with links to sections on the page.
22. Create styles.css and link it from the document head.
23. Style the page using element, class and ID selectors. Add spacing, typography, borders and a background.
24. Use Developer Tools to inspect computed styles and diagnose one deliberately introduced layout issue.

Worked Example / Reference Pattern

The following is a learning example. Adapt it to the task requirements and the project setup used in your class.

```
<form>
  <label for="student-name">Name</label>
  <input id="student-name" name="studentName" type="text" required>
```

```
  <label for="student-email">Email</label>
  <input id="student-email" name="email" type="email" required>

  <button type="submit">Save profile</button>
</form>

/* styles.css */
* { box-sizing: border-box; }
body { font-family: Arial, sans-serif; margin: 0; padding: 1rem; }
form { max-width: 32rem; display: grid; gap: .75rem; }
input { padding: .65rem; border: 1px solid #777; }
button { padding: .7rem 1rem; cursor: pointer; }
```

Independent Lab Exercises

25. Add a table caption and appropriate column headings.
26. Use :focus-visible to make keyboard focus easy to see.
27. Create a class selector for a reusable card and an ID selector for one unique section.
28. Change padding and margin separately and observe how the element's space changes.
29. Inspect a low-contrast text example and improve readability.

Verification Checklist

- Every form control has an associated label.
- The form uses suitable input types and required attributes where appropriate.
- The table has meaningful header cells.
- The page can be navigated with the keyboard.
- The CSS file is correctly linked and the box model is understood.

Lab Record and Reflection

Record item	Student notes
What I implemented	
One issue or error I encountered	
How I investigated and fixed it	
How I verified the result	
What I can now explain or modify independently	

AI-ASSISTED LAB ACTIVITY

AI Activity 2 - AI Accessibility and CSS Review

Copyable prompt - replace bracketed text with your own work:

> Review the following HTML form and CSS as a front-end code reviewer. Identify accessibility problems and CSS problems separately. For each issue, explain its impact and suggest the smallest correction. Do not rewrite the whole page. Check label association, keyboard focus, heading structure, table headers, selector specificity and the CSS box model. Here is my code: [paste your code].

Procedure

1. Build your form and table first without AI.
2. Paste your code into an AI assistant and request an accessibility/CSS review using the prompt.
3. Choose at least three findings. Verify each one in the browser or Developer Tools before changing code.
4. Ask AI to explain the difference between margin and padding using one element from your page.
5. Use keyboard-only navigation to test the corrected form and record the result.

Required evidence / submission

• Before/after code for at least three corrections.

• A short verification note for each correction.

• One example of AI advice that needed clarification or was not applicable.

Verification rule: run the code, test the behavior and check important claims against browser evidence or official documentation. You remain responsible for understanding and modifying the submitted work.

Lab 3 - Flexbox, Grid, Responsive Design, DevTools and MP1

Lab number: 03 | Course: CSE-2340 Software Development 1 | Syllabus week: 3

Learning Objectives

- Choose Flexbox or Grid based on the layout requirement.
- Apply a mobile-first responsive approach.
- Use DevTools to inspect viewport behavior and CSS rules.
- Build and deploy the MP1 semantic responsive page.

Concept Notes

Flexbox - A one-dimensional layout system useful for arranging items in a row or column.

Grid - A two-dimensional layout system useful for defining rows and columns together.

Mobile-first design - Start with a usable narrow-screen layout and progressively enhance it for wider viewports.

Media queries - Apply CSS conditionally based on viewport or device characteristics.

Specificity and cascade - Understand why one rule overrides another and use DevTools to inspect the winning declaration.

Required Tools / Setup

- VS Code, a modern browser and browser Developer Tools.
- The project files and tools required for this week's topic.
- Git/GitHub for version-controlled work; use the course repository workflow.

Guided Laboratory Procedure

30. Build a page with a header, navigation, main content and a sidebar/card region.
31. Use Flexbox for the navigation row and Grid for the main content arrangement.
32. Start with a single-column layout. Add a media query for wider screens.
33. Open DevTools responsive mode. Test narrow, medium and wide widths.
34. Inspect overflowing elements, text wrapping and image dimensions. Correct the cause rather than hiding all overflow.
35. Deploy the completed page using an instructor-approved static hosting workflow and test the public URL.

Worked Example / Reference Pattern

The following is a learning example. Adapt it to the task requirements and the project setup used in your class.

```
* { box-sizing: border-box; }
body { margin: 0; font-family: Arial, sans-serif; }
.page { display: grid; grid-template-columns: 1fr; gap: 1rem; padding: 1rem; }
.nav { display: flex; flex-wrap: wrap; gap: 1rem; }
```

```
.card-grid { display: grid; grid-template-columns: 1fr; gap: 1rem; }
.card { padding: 1rem; border: 1px solid #777; }
img { max-width: 100%; height: auto; }

@media (min-width: 48rem) {
  .page { grid-template-columns: 2fr 1fr; }
  .card-grid { grid-template-columns: repeat(2, minmax(0, 1fr)); }
}
```

Independent Lab Exercises

36. Create at least three content cards and make their layout responsive.
37. Make navigation wrap or otherwise remain usable at narrow widths.
38. Use a pseudo-class such as :hover or :focus-visible appropriately.
39. Find and fix one overflow issue using DevTools.
40. Deploy MP1 and prepare to explain the semantic structure, layout choices and responsive behavior.

Verification Checklist

- No horizontal scrolling is introduced at the tested viewport widths unless the content genuinely requires it.
- Images scale within their containers.
- Navigation remains usable.
- The deployed URL loads and matches the local version.
- You can explain why Flexbox or Grid was selected for each region.

Lab Record and Reflection

Record item	Student notes
What I implemented	
One issue or error I encountered	
How I investigated and fixed it	
How I verified the result	
What I can now explain or modify independently	

Assessment Checkpoint - MP1

Scheduled evaluation: Week 3 C4 | Marks: 5 | Deliverable: Semantic HTML + CSS + responsive layout + deployment.

Evaluation is individual. Be ready to demonstrate the working feature, explain an implementation decision and make a small live modification.

AI-ASSISTED LAB ACTIVITY

AI Activity 3 - Generate Responsive Test Cases, Then Fix the Layout

Copyable prompt - replace bracketed text with your own work:

> Act as a front-end QA reviewer. I have a page built with semantic HTML, Flexbox and CSS Grid. Create a responsive test matrix for narrow, medium and wide viewports. Include navigation, cards, images, text overflow and keyboard focus. Do not assume the page is responsive. Give me observable pass/fail criteria, then wait for my screenshots or observations before suggesting fixes.

Procedure

1. Ask AI to create a responsive test matrix before testing your page.

2. Test the page in browser responsive mode at three widths selected by you.

3. Record actual results; do not mark a test as passed based only on AI's prediction.

4. Share one actual failure and ask AI for two possible causes. Use DevTools to determine the real cause.

5. Complete MP1 and ask AI to review the README and deployment checklist; verify the live URL yourself.

Required evidence / submission

• Responsive test matrix with actual results.

• One defect report and evidence of the fix.

• MP1 deployed URL and a short explanation of the layout decisions.

Verification rule: run the code, test the behavior and check important claims against browser evidence or official documentation. You remain responsible for understanding and modifying the submitted work.

Lab 4 - Design Handoff, CSS Positioning and Tailwind CSS v4

Lab number: 04 | Course: CSE-2340 Software Development 1 | Syllabus week: 4

Learning Objectives

- Read measurements, spacing and assets from a prepared Figma/Pixso design.
- Use relative, absolute, fixed and sticky positioning appropriately.
- Set up Tailwind CSS v4 using the current official setup instructions supplied by the instructor.
- Recreate and modify a responsive interface using utility classes.

Concept Notes

Design handoff - A design handoff communicates dimensions, spacing, typography, colors and exported assets to developers.

Positioning - CSS positioning changes how an element participates in normal document flow and how it is located.

z-index - Stacking order determines which positioned elements appear in front when stacking contexts apply.

Utility-first CSS - Tailwind utility classes apply individual styling decisions directly in markup.

Required Tools / Setup

- VS Code, a modern browser and browser Developer Tools.
- The project files and tools required for this week's topic.
- Git/GitHub for version-controlled work; use the course repository workflow.

Guided Laboratory Procedure

41. Open the prepared Figma/Pixso frame and record the page width, spacing, font sizes, colors and assets needed.
42. Create the static HTML structure before adding detailed styling.
43. Implement a positioned badge or overlay using a positioned parent and an absolutely positioned child.
44. Implement a sticky header or fixed notice only when it serves the design requirement.
45. Set up Tailwind CSS v4 according to the official setup instructions used in class. Verify that generated utility styles are applied.
46. Rebuild a responsive card or landing-page section using spacing, typography, flex/grid and responsive utilities.

Worked Example / Reference Pattern

The following is a learning example. Adapt it to the task requirements and the project setup used in your class.

```
<!-- Illustrative Tailwind utility usage -->
<main class="mx-auto max-w-5xl p-4">
  <section class="grid grid-cols-1 gap-4 md:grid-cols-2">
    <article class="rounded-lg border p-5 shadow-sm">
      <h2 class="text-xl font-semibold">Course Dashboard</h2>
      <p class="mt-2 text-sm">Track your learning progress.</p>
```

```
      <a class="mt-4 inline-block underline" href="#details">View details</a>
    </article>
  </section>
</main>
```

Independent Lab Exercises

47. Recreate a supplied design region as closely as practical using the supplied measurements.
48. Create a card with an overlapping label using positioning.
49. Change the card layout at a wider breakpoint.
50. Modify spacing and typography without adding arbitrary one-off CSS for every element.
51. Compare the implementation with the design and record two visible differences.

Verification Checklist

- The chosen positioning mode matches the intended behavior.
- The absolute element is anchored to the intended containing block.
- Tailwind classes compile and render in the running project.
- The layout changes at the intended breakpoint.
- The page remains readable and usable at narrow widths.

Lab Record and Reflection

Record item	Student notes
What I implemented	
One issue or error I encountered	
How I investigated and fixed it	
How I verified the result	
What I can now explain or modify independently	

AI-ASSISTED LAB ACTIVITY

AI Activity 4 - Convert a Design Brief into Tailwind Tasks

Copyable prompt - replace bracketed text with your own work:

> I am implementing a supplied Figma/Pixso design using HTML and Tailwind CSS v4. Help me convert the design measurements and visual requirements I provide into an implementation checklist. Group the checklist into structure, spacing, typography, positioning and responsive behavior. Do not invent measurements or assets that I have not supplied. For each requirement, suggest the relevant CSS concept or Tailwind utility category.

Procedure

1. Inspect the supplied design and record real dimensions, colors, spacing and assets.
2. Ask AI to organize your notes into a checklist. Compare the checklist with the design and remove invented details.
3. Implement one card or page section using Tailwind CSS v4 as configured in class.
4. Ask AI to explain a positioning or z-index issue from your implementation; test the explanation with a small experiment.
5. Request a review of your responsive class choices and verify them at narrow and wide widths.

Required evidence / submission

• Design measurement notes and AI-generated checklist.

• Implemented Tailwind section.

• A short comparison listing two visual differences still remaining.

Verification rule: run the code, test the behavior and check important claims against browser evidence or official documentation. You remain responsible for understanding and modifying the submitted work.

Lab 5 - Responsive Deployment, JavaScript Fundamentals and MP2

Lab number: 05 | Course: CSE-2340 Software Development 1 | Syllabus week: 5

Learning Objectives

- Deploy a responsive page and verify its public URL.
- Use variables, primitive values and template literals.
- Use arrays, objects and functions to model and process student/course data.
- Validate inputs and display calculated results.

Concept Notes

Variables - Use const for bindings that are not reassigned and let for bindings that must be reassigned.

Primitive values - Common primitives include string, number, boolean, null, undefined, bigint and symbol.

Arrays - Arrays store ordered collections and use zero-based indexes.

Objects - Objects group related values under named properties.

Functions - Functions package reusable behavior and may accept parameters and return values.

Required Tools / Setup

- VS Code, a modern browser and browser Developer Tools.
- The project files and tools required for this week's topic.
- Git/GitHub for version-controlled work; use the course repository workflow.

Guided Laboratory Procedure

52. Check the responsive page from the previous lab and deploy it if not already deployed.
53. Create a JavaScript file and connect it to the HTML page using a script element.
54. Declare student name, course marks and maximum marks using suitable values.
55. Create a function that calculates a percentage and returns it.
56. Use a template literal to display a formatted result in the page.
57. Create an array of course records and an object for one student. Inspect and update their properties.

Worked Example / Reference Pattern

The following is a learning example. Adapt it to the task requirements and the project setup used in your class.

```
const student = { name: "Amina", marks: [78, 85, 91] };

function calculateAverage(values) {
  if (values.length === 0) return 0;
```

```
  const total = values.reduce((sum, value) => sum + value, 0);
  return total / values.length;
}

const average = calculateAverage(student.marks);
const message = `${student.name}'s average is ${average.toFixed(2)}`;
console.log(message);
```

Independent Lab Exercises

58. Build a student performance calculator with input validation.
59. Reject missing values and values outside the allowed range defined in your task.
60. Add at least three student records and calculate an average for each.
61. Display a clear message for an empty list.
62. Inspect and explain an AI-assisted implementation before modifying it.

Verification Checklist

- The calculation returns the expected value for known test data.
- An empty array does not cause an invalid division result.
- Invalid input is not silently treated as a valid mark.
- The displayed result is formatted clearly.
- MP2 is a deployed responsive design using Tailwind CSS.

Lab Record and Reflection

Record item	Student notes
What I implemented	
One issue or error I encountered	
How I investigated and fixed it	
How I verified the result	
What I can now explain or modify independently	

Assessment Checkpoint - MP2

Scheduled evaluation: Week 5 C4 | Marks: 5 | Deliverable: Own responsive design + Tailwind + deployment.

Evaluation is individual. Be ready to demonstrate the working feature, explain an implementation decision and make a small live modification.

AI-ASSISTED LAB ACTIVITY

AI Activity 5 - Use AI as a JavaScript Tutor, Not a Code Copier

Copyable prompt - replace bracketed text with your own work:

> Teach me how to use JavaScript variables, primitive values, arrays, objects and functions through a student marks calculator. Give me one small task at a time. Do not reveal the solution until I attempt it. After I submit my code, review it for correctness, edge cases and readability, and ask me to explain the result.

Procedure

1. Ask AI to create three progressively harder exercises using arrays of student marks.

2. Solve the first exercise yourself. Request a hint only if stuck; then compare your solution with the suggested approach.

3. Ask AI to generate test inputs for empty arrays, zero marks, maximum marks and invalid values.

4. Run the tests and record actual output. Correct any defect you find.

5. Complete MP2, then ask AI to review the deployed page against the task requirements without accepting unverified claims.

Required evidence / submission

• Your own initial solution and improved solution.

• Test cases with expected and actual output.

• One paragraph explaining why the final code is correct.

Verification rule: run the code, test the behavior and check important claims against browser evidence or official documentation. You remain responsible for understanding and modifying the submitted work.

Lab 6 - DOM Selection, Dynamic Elements and Event Handling

Lab number: 06 | Course: CSE-2340 Software Development 1 | Syllabus week: 6

Learning Objectives

- Select elements using DOM query methods.
- Update text, attributes and CSS classes through JavaScript.
- Create and append elements dynamically.
- Handle user actions using addEventListener and preventDefault.

Concept Notes

DOM - The Document Object Model represents the page as a tree of objects that scripts can inspect and modify.

Selectors - querySelector returns the first matching element; querySelectorAll returns a collection of matches.

classList - classList provides methods such as add, remove, toggle and contains.

Events - Events represent user or browser actions such as clicks, input, submit and key presses.

preventDefault - This method prevents a browser's default action when the application needs to handle the behavior itself.

Required Tools / Setup

- VS Code, a modern browser and browser Developer Tools.
- The project files and tools required for this week's topic.
- Git/GitHub for version-controlled work; use the course repository workflow.

Guided Laboratory Procedure

63. Create a task manager interface with an input, an Add button and an empty task list.
64. Select the form, input and list using querySelector.
65. Listen for the form submit event and call preventDefault.
66. Create a list item with createElement; set its text safely using textContent.
67. Add Complete and Delete controls. Use event listeners to update or remove the item.
68. Test empty input, repeated tasks, long text and rapid user interaction.

Worked Example / Reference Pattern

The following is a learning example. Adapt it to the task requirements and the project setup used in your class.

```
const form = document.querySelector("#task-form");
const input = document.querySelector("#task-input");
const list = document.querySelector("#task-list");
```

```
form.addEventListener("submit", (event) => {
  event.preventDefault();
  const title = input.value.trim();
  if (!title) return;

  const item = document.createElement("li");
  item.textContent = title;
  item.addEventListener("click", () => {
    item.classList.toggle("completed");
  });
  list.appendChild(item);
  input.value = "";
});
```

Independent Lab Exercises

69. Extend the task manager with a delete action.
70. Add a completed style using classList.
71. Add an item counter that updates when tasks are added or removed.
72. Build a small quiz or form interaction using events.
73. Ask a partner to introduce one new requirement and implement it without replacing the entire application.

Verification Checklist

- The page does not reload unexpectedly when the form is submitted.
- Whitespace-only input is handled.
- New elements appear in the correct location.
- Complete and delete actions affect only the intended task.
- No console error appears during the tested workflow.

Lab Record and Reflection

Record item	Student notes
What I implemented	
One issue or error I encountered	
How I investigated and fixed it	
How I verified the result	
What I can now explain or modify independently	

AI-ASSISTED LAB ACTIVITY

AI Activity 6 - AI-Assisted DOM Debugging

Copyable prompt - replace bracketed text with your own work:

> I am building a task manager using vanilla JavaScript and the DOM. Review this code and help me debug it. First identify the likely cause of the issue, explain how I can inspect it in DevTools, and give me one hint. Do not rewrite the entire application. My observed problem is: [describe the actual problem]. My code is: [paste relevant code].

Procedure

1. Build the task manager using querySelector, createElement, appendChild and addEventListener.
2. Create one real defect, such as an incorrect selector or a missing preventDefault call, and observe the failure.
3. Ask AI for a debugging hint using the prompt. Follow the suggested inspection steps in DevTools.
4. Fix the defect yourself and test empty input, valid input, repeated input and delete/complete behavior.
5. Ask AI to review whether your code safely handles user-provided text; verify that textContent is used where appropriate.

Required evidence / submission

• A short bug report: steps to reproduce, expected result, actual result and root cause.

• The corrected code and test results.

• A note explaining why the final fix is safe and appropriate.

Verification rule: run the code, test the behavior and check important claims against browser evidence or official documentation. You remain responsible for understanding and modifying the submitted work.

Lab 7 - Array Methods and Mid-term Practical Preparation

Lab number: 07 | Course: CSE-2340 Software Development 1 | Syllabus week: 7

Learning Objectives

- Use map, filter, forEach and find for common UI data tasks.
- Render a searchable or filterable list.
- Combine arrays, DOM and event handling.
- Prepare for the individual 100-minute mid-term practical.

Concept Notes

map - Creates a new array by transforming every element.

filter - Creates a new array containing elements that satisfy a condition.

forEach - Runs a callback for each element when a returned transformed array is not needed.

find - Returns the first matching element or undefined.

Search/filter UI - The interface takes user input, filters a data collection and renders the matching records.

Required Tools / Setup

- VS Code, a modern browser and browser Developer Tools.
- The project files and tools required for this week's topic.
- Git/GitHub for version-controlled work; use the course repository workflow.

Guided Laboratory Procedure

74. Create an array of product or student objects with at least five records.
75. Use map to extract a list of names or titles.
76. Use filter to select records that meet a condition such as category or score.
77. Use find to retrieve one record by an identifier.
78. Create a search input and update the displayed list when its value changes.
79. Practise a timed integrated task using HTML, CSS, JavaScript and DOM without using React or Firebase.

Worked Example / Reference Pattern

The following is a learning example. Adapt it to the task requirements and the project setup used in your class.

```
const students = [
  { id: 1, name: "Amina", score: 82 },
  { id: 2, name: "Rafi", score: 67 },
  { id: 3, name: "Nadia", score: 91 }
```

```
];

const names = students.map(student => student.name);
const passed = students.filter(student => student.score >= 70);
const selected = students.find(student => student.id === 2);

console.log(names, passed, selected);
```

Independent Lab Exercises

80. Add a search that matches names without case sensitivity.
81. Add a filter for students who passed a defined threshold.
82. Display a helpful message when no record matches.
83. Practise rebuilding a small application from a written problem statement in a limited time.
84. Explain the data flow from the input event to filtering and DOM rendering.

Verification Checklist

- map returns a new transformed array.
- filter does not modify the original array.
- find handles a missing identifier.
- Search handles empty input and no matches.
- Your mid-term practice uses only the pre-midterm syllabus scope.

Lab Record and Reflection

Record item	Student notes
What I implemented	
One issue or error I encountered	
How I investigated and fixed it	
How I verified the result	
What I can now explain or modify independently	

AI-ASSISTED LAB ACTIVITY

AI Activity 7 - Array Method Practice with AI-Generated Test Data

Copyable prompt - replace bracketed text with your own work:

> Create a JavaScript practice challenge using an array of student objects. The task must require map, filter, forEach and find. Give me the problem statement and sample input/output first, but do not provide the solution. After I attempt it, review my code and generate edge-case tests.

Procedure

1. Ask AI for a data-processing challenge and solve it independently.
2. Ask AI to generate at least five edge cases, including no matching student and an empty array.
3. Run each case and record the actual result.
4. Ask AI to compare the suitability of map, filter, forEach and find for your solution; explain the choice in your own words.
5. Use AI to create a short mock practical task, but practise under timed conditions without AI to prepare for the mid-term.

Required evidence / submission

• Problem statement, your solution and AI-generated test cases.

• Expected-versus-actual test results.

• A short explanation of which array method was used and why.

Verification rule: run the code, test the behavior and check important claims against browser evidence or official documentation. You remain responsible for understanding and modifying the submitted work.

Lab 8 - Modern JavaScript, Refactoring, GitHub Workflow and MP3

Lab number: 08 | Course: CSE-2340 Software Development 1 | Syllabus week: 8

Learning Objectives

- Read and refactor arrow functions and modern object/array syntax.
- Use spread/rest, destructuring and optional chaining appropriately.
- Explain truthy/falsy values and strict equality.
- Use branches, commits and pull requests in GitHub.
- Complete MP3 with an individual explanation and live modification.

Concept Notes

Arrow functions - Provide concise function syntax; lexical this behavior differs from ordinary functions.

Destructuring - Extract values from arrays or object properties into variables.

Spread and rest - Spread expands iterable/object values; rest collects remaining values.

Optional chaining - Safely accesses a nested property when an intermediate value may be null or undefined.

Strict equality - === compares without automatic type coercion; it is generally preferred for predictable comparisons.

Pull request - A request to review and integrate a branch's changes into another branch.

Required Tools / Setup

- VS Code, a modern browser and browser Developer Tools.
- The project files and tools required for this week's topic.
- Git/GitHub for version-controlled work; use the course repository workflow.

Guided Laboratory Procedure

85. Open an existing JavaScript application and identify repeated or difficult-to-read code.
86. Refactor suitable functions into arrow functions while preserving behavior.
87. Use destructuring and optional chaining where they improve clarity and safety.
88. Create a feature branch and make a focused change.
89. Push the branch and open a pull request. Describe the change and how you tested it.
90. For MP3, prepare either a DOM application or the assigned debug-an-existing-codebase task.

Worked Example / Reference Pattern

The following is a learning example. Adapt it to the task requirements and the project setup used in your class.

```
const user = { profile: { name: "Nadia" } };
const displayName = user.profile?.name ?? "Guest";
```

```
const scores = [65, 80, 92];
const [first, ...remaining] = scores;
const updatedScores = [...scores, 88];

const formatUser = ({ profile }) => profile?.name ?? "Guest";
console.log(displayName, first, remaining, updatedScores, formatUser(user));
```

Independent Lab Exercises

91. Find and correct a defect in the starter code provided by the instructor.
92. Add one feature without breaking existing behavior.
93. Create a branch named feature/your-feature and open a pull request.
94. Review a teammate's pull request and leave a specific, constructive comment.
95. Prepare to explain one function and modify it live during MP3 evaluation.

Verification Checklist

- Refactoring preserves expected output.
- Optional chaining prevents an access error for missing nested data.
- The feature branch contains the intended changes.
- The pull request describes the change and testing performed.
- You can explain the DOM logic or debugging resolution independently.

Lab Record and Reflection

Record item	Student notes
What I implemented	
One issue or error I encountered	
How I investigated and fixed it	
How I verified the result	
What I can now explain or modify independently	

Assessment Checkpoint - MP3

Scheduled evaluation: Week 8 C4 | Marks: 7 | Deliverable: DOM application or debugging task.

Evaluation is individual. Be ready to demonstrate the working feature, explain an implementation decision and make a small live modification.

AI-ASSISTED LAB ACTIVITY

AI Activity 8 - Refactor and Review a Codebase with AI

Copyable prompt - replace bracketed text with your own work:

> Act as a JavaScript code reviewer. Review the code I provide for readability, modern JavaScript usage and possible defects. Identify opportunities involving arrow functions, destructuring, spread/rest, optional chaining and strict equality only where they genuinely improve the code. Do not refactor everything. Explain each proposed change and preserve existing behavior. Code: [paste your code].

Procedure

1. Ask AI to review a small existing DOM application or the instructor's debugging starter.
2. Choose two suggested refactorings and inspect the before/after behavior.
3. Ask AI to explain one confusing expression, such as optional chaining or destructuring, using your code.
4. Request a pull-request description summarizing the change and tests. Edit it so it accurately describes your actual work.
5. Open a branch and PR. Ask AI to draft review questions, then personally inspect the changed files.

Required evidence / submission

• Before/after diff or code excerpts.

• Evidence that existing behavior still works.

• Git branch, PR description and a reflection on one AI suggestion you rejected or modified.

• For MP3, demonstrate and explain the actual DOM application or debugging task independently.

Verification rule: run the code, test the behavior and check important claims against browser evidence or official documentation. You remain responsible for understanding and modifying the submitted work.

Lab 9 - HTTP, Asynchronous JavaScript, Fetch, JSON and Project Planning

Lab number: 09 | Course: CSE-2340 Software Development 1 | Syllabus week: 9

Learning Objectives

- Explain HTTP/HTTPS and the purpose of an API endpoint.
- Distinguish synchronous and asynchronous execution.
- Use Promises, async/await and try/catch/finally.
- Read API documentation and handle invalid or failed responses.
- Prepare a final group project proposal using user stories and a backlog.

Concept Notes

API - An interface that allows software systems to exchange data or request functionality.

JSON - A text-based data format commonly used to represent API request and response bodies.

Promise - An object representing the eventual completion or failure of an asynchronous operation.

async/await - Syntax that makes Promise-based asynchronous code easier to read.

HTTP status - A response code indicating the outcome category of a request.

UI states - A data-driven interface should communicate loading, success, empty and error conditions.

Required Tools / Setup

- VS Code, a modern browser and browser Developer Tools.
- The project files and tools required for this week's topic.
- Git/GitHub for version-controlled work; use the course repository workflow.

Guided Laboratory Procedure

96. Select an unfamiliar public API suitable for a simple classroom demonstration. Read its documentation before coding.
97. Identify the endpoint, required parameters, expected JSON structure and possible failure conditions.
98. Create a button that starts a fetch request.
99. Use async/await inside a try/catch/finally structure. Display loading feedback before the response arrives.
100. Check response.ok before treating the response as successful. Parse JSON and render selected fields.
101. Inspect the Network tab and deliberately test a failed request or invalid endpoint.
102. For the final group project, write a short problem statement, target users, user stories and a prioritized backlog for instructor review.

Worked Example / Reference Pattern

The following is a learning example. Adapt it to the task requirements and the project setup used in your class.

```
async function loadItems() {
  const status = document.querySelector("#status");
  const output = document.querySelector("#output");
  status.textContent = "Loading...";
  output.replaceChildren();

  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/posts");
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    const items = await response.json();

    if (items.length === 0) {
      status.textContent = "No records found.";
      return;
    }
    status.textContent = "Loaded successfully.";
    items.slice(0, 5).forEach(item => {
      const article = document.createElement("article");
      article.textContent = item.title;
      output.appendChild(article);
    });
  } catch (error) {
    status.textContent = `Could not load data: ${error.message}`;
  }
}
```

Independent Lab Exercises

103. Add a search/filter function to the API-driven interface.
104. Handle an empty result separately from a failed request.
105. Inspect the request URL, status and response body in Network tools.
106. Write at least five user stories for your final project using the format: As a [user], I want [capability], so that [benefit].
107. Prioritize the backlog and identify a realistic first implementation increment.

Verification Checklist

- The UI indicates when a request is in progress.
- Non-success HTTP responses are handled.
- The response is parsed as JSON only after a successful response check.
- Network errors do not leave the interface permanently showing Loading.
- The proposal includes a defined user problem and scope.

Lab Record and Reflection

Record item	Student notes
What I implemented	
One issue or error I encountered	
How I investigated and fixed it	
How I verified the result	
What I can now explain or modify independently	

AI-ASSISTED LAB ACTIVITY

AI Activity 9 - API Documentation Interpreter and Async Code Assistant

Copyable prompt - replace bracketed text with your own work:

> Help me understand this API documentation: [paste the relevant public documentation excerpt]. Extract the endpoint, HTTP method, parameters, expected JSON shape and possible error responses. Do not invent undocumented behavior. Then guide me in writing a fetch request with async/await, loading state, error handling and an empty state. Explain each part before showing a small example.

Procedure

1. Select an API and read its official documentation. Ask AI to summarize only the excerpt or URL content you provide.
2. Check every endpoint, parameter and response field against the official documentation.
3. Ask AI to explain the difference between a rejected fetch and a non-2xx HTTP response.
4. Implement a fetch workflow with try/catch/finally and response.ok handling.
5. Ask AI to review your final group project's user stories for ambiguity and testability; discuss and revise them with your team.

Required evidence / submission

• API evidence sheet: endpoint, method, parameters and response fields with source references.

• Working fetch code and test evidence for success and failure.

• Final project user stories and revised backlog.

Verification rule: run the code, test the behavior and check important claims against browser evidence or official documentation. You remain responsible for understanding and modifying the submitted work.

Lab 10 - API-Driven Interfaces, Search, Filtering, localStorage and MP4

Lab number: 10 | Course: CSE-2340 Software Development 1 | Syllabus week: 10

Learning Objectives

- Render API data dynamically in a front-end interface.
- Implement loading, error and empty states.
- Add search and filtering to remote data.
- Store and retrieve simple values and objects using localStorage.
- Deploy the MP4 application.

Concept Notes

Dynamic rendering - The page is updated from data rather than being manually hard-coded for each record.

localStorage - A browser storage mechanism that stores string values by key and persists across reloads for the same origin.

JSON.stringify / parse - Convert objects to JSON text before storage and parse JSON text after retrieval.

Persistence boundary - localStorage is browser-local and should not be treated as secure storage for secrets.

Required Tools / Setup

- VS Code, a modern browser and browser Developer Tools.
- The project files and tools required for this week's topic.
- Git/GitHub for version-controlled work; use the course repository workflow.

Guided Laboratory Procedure

108. Reuse the API interface from Lab 9 or start from the instructor's starter project.
109. Separate request status from rendered data so loading, error, empty and success states can be represented clearly.
110. Add search and a filter control. Derive visible records from the fetched dataset.
111. Add a Save button for a selected item or a simple cart/saved-item list.
112. Serialize the saved object/list with JSON.stringify before writing to localStorage.
113. On page load, read the stored string, parse it safely and restore the interface. Handle missing or invalid stored data.
114. Deploy the application and test persistence on the deployed origin.

Worked Example / Reference Pattern

The following is a learning example. Adapt it to the task requirements and the project setup used in your class.

```
const STORAGE_KEY = "sd1-saved-items";

function readSavedItems() {
```

```
  try {
    return JSON.parse(localStorage.getItem(STORAGE_KEY)) ?? [];
  } catch {
    return [];
  }
}

function saveItems(items) {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(items));
}

let savedItems = readSavedItems();
savedItems.push({ id: 7, title: "Saved article" });
saveItems(savedItems);
```

Independent Lab Exercises

115. Add a remove-from-saved-items action.
116. Prevent duplicate saved records using a stable identifier.
117. Show a meaningful message when the saved list is empty.
118. Test malformed stored JSON using DevTools and ensure the app recovers gracefully.
119. Deploy MP4 and verify API behavior and local persistence.

Verification Checklist

- The interface distinguishes loading, error, empty and success states.
- Search/filter does not mutate the original response unnecessarily.
- Saved data remains after a page reload.
- Invalid stored JSON does not crash the application.
- No secret or credential is stored in localStorage.

Lab Record and Reflection

Record item	Student notes
What I implemented	
One issue or error I encountered	
How I investigated and fixed it	
How I verified the result	
What I can now explain or modify independently	

Assessment Checkpoint - MP4

Scheduled evaluation: Week 10 C4 | Marks: 7 | Deliverable: API application + localStorage + loading/error states + deployment.

Evaluation is individual. Be ready to demonstrate the working feature, explain an implementation decision and make a small live modification.

AI-ASSISTED LAB ACTIVITY

AI Activity 10 - Build and Test API UI States; Review localStorage

Copyable prompt - replace bracketed text with your own work:

> Review my API-driven JavaScript interface. Check whether it clearly handles loading, success, empty results and errors. Then review my search/filter and localStorage logic for duplicate items, invalid stored JSON and missing values. Give findings in priority order and suggest focused fixes. Do not claim that code works unless you can identify a test I should run. Code: [paste relevant code].

Procedure

1. Implement the API interface and localStorage persistence before requesting a review.
2. Ask AI to identify missing UI states and create a test checklist.
3. Test success, empty response, failed request and malformed localStorage data.
4. Ask AI to inspect the storage logic for duplicate records and parsing errors. Apply only verified fixes.
5. Complete MP4 and ask AI to review the README and test evidence; verify the deployment yourself.

Required evidence / submission

• Test matrix covering loading, success, empty, error and persistence.

• One example of a defect found during AI review and the verified correction.

• MP4 deployed URL and README.

Verification rule: run the code, test the behavior and check important claims against browser evidence or official documentation. You remain responsible for understanding and modifying the submitted work.

Lab 11 - Vite, React Components, JSX, Props, Lists, State and Effects

Lab number: 11 | Course: CSE-2340 Software Development 1 | Syllabus week: 11

Learning Objectives

- Create a React project using Vite.
- Explain JSX, components and component boundaries.
- Pass data through props and render lists conditionally.
- Handle events and update state with useState.
- Use useEffect for a suitable side effect or API request.

Concept Notes

Vite - A development/build tool used to scaffold and run the React project in this course.

Component - A reusable unit of UI that returns React elements.

JSX - A syntax extension that describes UI structure within JavaScript.

Props - Inputs passed from a parent component to a child component.

State - Data owned by a component that can change and trigger a re-render.

useEffect - A React Hook for synchronizing a component with an external system, such as a network request.

Required Tools / Setup

- VS Code, a modern browser and browser Developer Tools.
- The project files and tools required for this week's topic.
- Git/GitHub for version-controlled work; use the course repository workflow.

Guided Laboratory Procedure

120. Create a Vite React project using the command and template demonstrated by the instructor.
121. Run the development server and inspect the starter application.
122. Create a StudentCard component that accepts name, department and status as props.
123. Create an array of student objects and render StudentCard components using map with a stable key.
124. Add conditional rendering for an empty list.
125. Add a button and useState to toggle a visible state or update a counter.
126. Use useEffect for a small API request or other appropriate external synchronization, with loading/error handling.

Worked Example / Reference Pattern

The following is a learning example. Adapt it to the task requirements and the project setup used in your class.

```
function StudentCard({ student }) {
  return (
    <article>
      <h2>{student.name}</h2>
      <p>{student.department}</p>
      <p>{student.active ? "Active" : "Inactive"}</p>
    </article>
  );
}

function App() {
  const students = [
    { id: 1, name: "Amina", department: "CSE", active: true },
    { id: 2, name: "Rafi", department: "CSE", active: false }
  ];

  return (
    <main>
      {students.length
        ? students.map(student =>
            <StudentCard key={student.id} student={student} />
          )
        : <p>No students available.</p>}
    </main>
  );
}
```

Independent Lab Exercises

127. Split the page into Header, StudentList and StudentCard components.
128. Pass a callback prop from a parent to a child and invoke it from a button.
129. Add a controlled search field using state.
130. Explain why a list key should be stable and unique.
131. Modify one component or state behavior without rewriting the entire app.

Verification Checklist

- The Vite development server starts successfully.
- Components render the expected props.
- Lists use stable keys.
- State changes update the interface.
- Effects are used for an appropriate external synchronization and do not create an unintended repeated request.

Lab Record and Reflection

Record item	Student notes
What I implemented	
One issue or error I encountered	
How I investigated and fixed it	
How I verified the result	
What I can now explain or modify independently	

AI-ASSISTED LAB ACTIVITY

AI Activity 11 - AI as a React Pair Programmer

Copyable prompt - replace bracketed text with your own work:

> Act as a React tutor. Help me split this interface into sensible components and identify which values should be props and which should be state. Do not introduce Context API or custom hooks. Stay within components, JSX, props, lists, conditional rendering, events, useState and useEffect. First propose a component tree and explain the data flow; wait for my approval before generating code.

Procedure

1. Build a small React page in Vite and sketch the component tree yourself.
2. Ask AI to review your component boundaries and explain the props/state distinction.
3. Implement the components and compare your structure with AI's suggestion.
4. Ask AI to inspect list keys and state updates. Verify the behavior in the running app.
5. Use AI to generate a short change request, then implement the requested change yourself.

Required evidence / submission

• Component tree and explanation of data flow.

• React source code with working props, list rendering and state interaction.

• A brief record of one AI suggestion accepted and one not used.

Verification rule: run the code, test the behavior and check important claims against browser evidence or official documentation. You remain responsible for understanding and modifying the submitted work.

Lab 12 - Immutable State, Lifting State, React Router and ShadCN/ui

Lab number: 12 | Course: CSE-2340 Software Development 1 | Syllabus week: 12

Learning Objectives

- Update arrays and objects in React state immutably.
- Lift shared state to a common parent when multiple components need it.
- Configure routes, nested layouts and navigation with React Router.
- Use dynamic route parameters and navigation hooks.
- Install/use the prepared ShadCN/ui components covered in the syllabus.

Concept Notes

Immutable update - Create a new array/object rather than mutating the existing state value directly.

Lifting state up - Move shared state to the closest common parent and pass values/callbacks through props.

Route - A mapping between a URL path and a rendered page/component.

Nested route - A child route rendered inside a parent layout, commonly through Outlet.

Dynamic parameter - A variable segment in a route path, accessed with useParams.

ShadCN/ui - A component approach used in the course's prepared setup for practical interface components.

Required Tools / Setup

- VS Code, a modern browser and browser Developer Tools.
- The project files and tools required for this week's topic.
- Git/GitHub for version-controlled work; use the course repository workflow.

Guided Laboratory Procedure

132. Add a list state to your React application. Implement add/remove/update operations by creating new arrays or objects.
133. Move shared state to the common parent if two components need to read or update it.
134. Install/configure React Router using the prepared course project setup.
135. Create Home, Students and Student Details routes. Add a shared layout with navigation links and Outlet.
136. Use a dynamic route such as /students/:studentId and read the parameter with useParams.
137. Use useNavigate for a button-driven navigation action. Add a not-found route and a loading state where appropriate.
138. Use the prepared ShadCN/ui setup to add a Button, Card, Dialog or Table component.

Worked Example / Reference Pattern

The following is a learning example. Adapt it to the task requirements and the project setup used in your class.

```
import { Routes, Route, Link, Outlet, useParams } from "react-router-dom";

function Layout() {
  return (
    <>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/students">Students</Link>
      </nav>
      <Outlet />
    </>
  );
}

function StudentDetails() {
  const { studentId } = useParams();
  return <h2>Student ID: {studentId}</h2>;
}

function AppRoutes() {
  return (
    <Routes>
      <Route element={<Layout />}>
        <Route path="/" element={<h1>Home</h1>} />
        <Route path="/students" element={<h1>Student List</h1>} />
        <Route path="/students/:studentId" element={<StudentDetails />} />
        <Route path="*" element={<h1>404 - Not Found</h1>} />
      </Route>
    </Routes>
  );
}
```

Independent Lab Exercises

139. Add a nested route for a student section.
140. Add a navigation action using useNavigate after a successful form submission.
141. Add a loading state to a route that depends on data.
142. Use a ShadCN/ui Dialog for a confirmation interaction or a Table for records.
143. Test direct URL entry, refresh and navigation between routes.

Verification Checklist

- State updates do not mutate previous state arrays/objects.
- Nested pages render inside the expected layout.
- Links navigate without a full-page reload in the SPA.
- Dynamic route parameters are read correctly.
- Unknown paths show a 404 page.
- The selected ShadCN/ui component renders and behaves as expected.

Lab Record and Reflection

Record item	Student notes
What I implemented	
One issue or error I encountered	
How I investigated and fixed it	
How I verified the result	
What I can now explain or modify independently	

AI-ASSISTED LAB ACTIVITY

AI Activity 12 - Route Planning and UI Component Selection with AI

Copyable prompt - replace bracketed text with your own work:

> I am building a React SPA for [project idea]. Based only on these requirements, propose a route table with path, page purpose and whether authentication is required. Then identify where a Button, Card, Dialog or Table from our prepared ShadCN/ui setup would be useful. Do not add features or libraries outside the course scope. Explain the reason for each choice.

Procedure

1. Write your route requirements before asking AI for a route table.
2. Compare AI's route proposal with your requirements and implement routes using React Router.
3. Ask AI to review direct URL access, nested layouts, dynamic parameters and the 404 route.
4. Ask for a ShadCN/ui component recommendation for one actual interaction. Implement it using the prepared setup.
5. Test every route manually, including refresh and unknown paths.

Required evidence / submission

• Route table and implemented React Router configuration.

• Evidence that direct navigation, nested routes and 404 handling were tested.

• One paragraph explaining why the selected ShadCN/ui component fits the interaction.

Verification rule: run the code, test the behavior and check important claims against browser evidence or official documentation. You remain responsible for understanding and modifying the submitted work.

Lab 13 - Controlled Forms, Validation, Reusable Components, Debugging and MP5

Lab number: 13 | Course: CSE-2340 Software Development 1 | Syllabus week: 13

Learning Objectives

- Build controlled forms using React state.
- Validate user input and display useful feedback.
- Create reusable components using props and children.
- Inspect and debug component, state and prop problems.
- Complete and deploy MP5.

Concept Notes

Controlled input - The displayed input value is driven by React state and updated through an event handler.

Validation - Check that input satisfies required conditions before accepting or processing it.

Reusable component - A component designed to serve multiple parts of an interface through configurable props.

children - A special prop that represents nested content placed between a component's opening and closing tags.

Debugging - Reproduce the problem, inspect state/props and component behavior, isolate the cause, change and retest.

Required Tools / Setup

- VS Code, a modern browser and browser Developer Tools.
- The project files and tools required for this week's topic.
- Git/GitHub for version-controlled work; use the course repository workflow.

Guided Laboratory Procedure

144. Create a form with name, email and course fields.
145. Use state variables to control the input values.
146. Handle changes through onChange and update the appropriate state.
147. Validate required fields and email format before submission.
148. Display field-level or form-level feedback. Do not rely only on color to communicate an error.
149. Extract a reusable FormField or Button component using props and children.
150. Use React Developer Tools or browser debugging to inspect an incorrect prop/state value.
151. Prepare MP5: a deployed React SPA with routing, reusable components and ShadCN/ui.

Worked Example / Reference Pattern

The following is a learning example. Adapt it to the task requirements and the project setup used in your class.

```
function ProfileForm() {
  const [name, setName] = React.useState("");
  const [email, setEmail] = React.useState("");
  const [error, setError] = React.useState("");

  function handleSubmit(event) {
    event.preventDefault();
    if (!name.trim() || !email.includes("@")) {
      setError("Enter a name and a valid email address.");
      return;
    }
    setError("");
    console.log({ name, email });
  }

  return (
    <form onSubmit={handleSubmit}>
      <label>Name
        <input value={name} onChange={e => setName(e.target.value)} />
      </label>
      <label>Email
        <input value={email} onChange={e => setEmail(e.target.value)} />
      </label>
      {error && <p role="alert">{error}</p>}
      <button type="submit">Save</button>
    </form>
  );
}
```

Independent Lab Exercises

152. Add a reset action that clears all controlled fields.
153. Prevent submission when required values are invalid.
154. Extract one reusable component and use it in at least two places.
155. Diagnose a deliberately incorrect prop or state update.
156. Prepare to explain routing, component reuse and a live modification during MP5 evaluation.

Verification Checklist

- Input values remain synchronized with state.
- The form does not submit when validation fails.
- Feedback is understandable and associated with the form.
- Reusable components accept and use props correctly.
- The deployed MP5 application has working routes and the required UI library feature.

Lab Record and Reflection

Record item	Student notes
What I implemented	
One issue or error I encountered	
How I investigated and fixed it	
How I verified the result	
What I can now explain or modify independently	

Assessment Checkpoint - MP5

Scheduled evaluation: Week 13 C4 | Marks: 8 | Deliverable: React SPA + routing + reusable components + ShadCN/ui + deployment.

Evaluation is individual. Be ready to demonstrate the working feature, explain an implementation decision and make a small live modification.

AI-ASSISTED LAB ACTIVITY

AI Activity 13 - AI Form Review, Validation and Debugging

Copyable prompt - replace bracketed text with your own work:

> Review my React controlled form. Check whether each input is controlled, validation is meaningful, errors are communicated accessibly, and reusable components have appropriate props/children. Identify defects and explain how I can reproduce each one. Do not replace my form with a completely different implementation. Code: [paste code].

Procedure

1. Build a controlled form and write down validation rules before consulting AI.
2. Ask AI to review the form using the prompt. Verify each finding by submitting valid and invalid data.
3. Ask AI to suggest reusable component boundaries; implement at least one reusable field or button component.
4. Use AI to generate a test matrix for blank, malformed and valid values. Run the tests yourself.
5. Complete MP5 and explain the routing, reusable component and ShadCN/ui feature without relying on AI during the explanation.

Required evidence / submission

• Validation rules and AI review findings.

• Test matrix with actual results.

• MP5 deployed URL and a short code walkthrough.

Verification rule: run the code, test the behavior and check important claims against browser evidence or official documentation. You remain responsible for understanding and modifying the submitted work.

Lab 14 - Firebase Authentication, Private Routes, Firestore and MP6

Lab number: 14 | Course: CSE-2340 Software Development 1 | Syllabus week: 14

Learning Objectives

- Create/configure a Firebase project and add the SDK to the application.
- Implement Google or email/password authentication flows covered in the syllabus.
- Track authentication state and protect private routes.
- Read and write documents in one Firestore collection.
- Use the supplied security-rules starter and protect configuration information.

Concept Notes

Firebase project - A project groups Firebase services and application configuration.

Authentication - Confirms the identity of a user through a supported sign-in method.

Auth state - The application needs to respond when a user signs in or signs out.

Private route - A route that is available only when the required authentication condition is satisfied.

Firestore collection - A collection groups documents; this lab uses one collection at the syllabus-defined basic read/write scope.

Configuration safety - Never publish private service credentials or commit secrets. Follow the supplied rules and instructor setup.

Required Tools / Setup

- VS Code, a modern browser and browser Developer Tools.
- The project files and tools required for this week's topic.
- Git/GitHub for version-controlled work; use the course repository workflow.

Guided Laboratory Procedure

157. Create or open the Firebase project specified for the lab. Register the web app and configure the SDK using the instructor-provided setup process.
158. Enable the authentication providers required for the exercise: Google and/or email/password.
159. Implement sign-in and sign-out controls. Display the current authentication state.
160. Implement registration/login if using email/password. Validate required fields and show errors.
161. Create a private route that redirects or displays an access message for unauthenticated users.
162. Create one Firestore collection. Implement the basic read and write operations demonstrated in class.
163. Apply the supplied security-rules starter. Do not replace it with unrestricted production rules.
164. Deploy MP6 and test authenticated and unauthenticated behavior.

Worked Example / Reference Pattern

The following is a learning example. Adapt it to the task requirements and the project setup used in your class.

```
// Illustrative authentication-state pattern.
// Use the Firebase app/auth initialization supplied in your lab project.
import { onAuthStateChanged } from "firebase/auth";

useEffect(() => {
  const unsubscribe = onAuthStateChanged(auth, (user) => {
    setCurrentUser(user);
    setAuthLoading(false);
  });
  return unsubscribe;
}, []);

// Illustrative basic Firestore write:
import { addDoc, collection } from "firebase/firestore";

async function addNotice() {
  await addDoc(collection(db, "notices"), {
    title: "Lab announcement",
    createdAt: new Date()
  });
}
```

Independent Lab Exercises

165. Add a clear sign-out action and verify the private route becomes unavailable.
166. Display an authentication loading state while the initial auth state is being resolved.
167. Write one document to the selected collection and read the collection back into the interface.
168. Handle a failed read/write with a visible message.
169. Check that no secret or private credential appears in source control.

Verification Checklist

- Sign-in and sign-out update the visible application state.
- Unauthenticated users cannot access the protected route.
- The application can add and retrieve a document in the selected collection.
- Errors are handled without crashing the page.
- The supplied security-rules starter is used and configuration is handled safely.

Lab Record and Reflection

Record item	Student notes
What I implemented	
One issue or error I encountered	
How I investigated and fixed it	
How I verified the result	
What I can now explain or modify independently	

Assessment Checkpoint - MP6

Scheduled evaluation: Week 14 C4 | Marks: 8 | Deliverable: Firebase Authentication + one Firestore collection + deployment.

Evaluation is individual. Be ready to demonstrate the working feature, explain an implementation decision and make a small live modification.

AI-ASSISTED LAB ACTIVITY

AI Activity 14 - Firebase Security and Auth Flow Review

Copyable prompt - replace bracketed text with your own work:

> Review my Firebase Authentication and Firestore implementation at a beginner course scope. Check the sign-in/sign-out flow, authentication loading state, protected route behavior, one-collection read/write and whether I have exposed secrets. Do not suggest Context API, advanced database design or unrestricted security rules. Identify what I must verify in Firebase Console and in the supplied security-rules starter. Code/configuration with secrets removed: [paste relevant material].

Procedure

1. Implement authentication and auth-state handling first. Never paste passwords, private keys or secret credentials into an AI assistant.

2. Ask AI to review the sign-in/sign-out and protected-route flow using redacted code.

3. Verify provider settings and project configuration directly in Firebase Console.

4. Implement one Firestore collection with basic read/write. Ask AI to help explain a permission error, but inspect the actual Firebase error and supplied rules.

5. Test unauthenticated access, successful sign-in, sign-out, read and write. Complete MP6.

Required evidence / submission

• Redacted AI review prompt and findings.

• Test evidence for auth and Firestore operations.

• MP6 deployed URL and a note confirming no secret was shared or committed.

Verification rule: run the code, test the behavior and check important claims against browser evidence or official documentation. You remain responsible for understanding and modifying the submitted work.

Lab 15 - Final Integration, Quality Assurance, Pull Request Review and Showcase

Lab number: 15 | Course: CSE-2340 Software Development 1 | Syllabus week: 15

Learning Objectives

- Integrate and test the final group application.
- Inspect responsive behavior and accessibility basics.
- Run a lightweight Lighthouse audit and interpret findings.
- Review pull requests and verify each member's contribution.
- Demonstrate the final application and complete the individual viva.

Concept Notes

Integration testing - Check that connected features work together, not only in isolation.

Accessibility review - Inspect structure, labels, keyboard use, focus and readable content.

Lighthouse - A browser auditing tool that provides diagnostic information about performance, accessibility, best practices and SEO.

Pull request review - Inspect the changes, understand their purpose and verify the proposed integration.

Individual ownership - A team deliverable still requires each member to demonstrate their own feature, branch, pull request, commits and understanding.

Required Tools / Setup

- VS Code, a modern browser and browser Developer Tools.
- The project files and tools required for this week's topic.
- Git/GitHub for version-controlled work; use the course repository workflow.

Guided Laboratory Procedure

170. Pull the latest approved project changes and verify the application starts from a clean setup using the README.
171. Walk through all required routes, authentication and Firestore operations.
172. Test the unauthenticated path, invalid input, empty data and failed network/service responses where applicable.
173. Inspect responsive behavior at narrow and wide viewports. Correct usability issues.
174. Run a lightweight Lighthouse audit and record important findings. Address practical issues within the project scope.
175. Review each open pull request. Check changed files, behavior and test evidence before approval/merge.

176. Verify README, deployed URL, feature completeness, member-owned feature, branch, pull request and at least eight meaningful commits per member.
177. Prepare a concise team demonstration. Each member should explain and modify their own feature during viva.

Worked Example / Reference Pattern

The following is a learning example. Adapt it to the task requirements and the project setup used in your class.

```
# Example README outline

# Project Name
## Problem and target users
## Features
## Technology stack
## Routes
## Authentication and protected areas
## Firestore collection and data operations
## Local setup
## Environment/configuration notes
## Deployment URL
## Team members and individual contributions
## AI assistance acknowledgement
```

Independent Lab Exercises

178. Prepare a feature-to-member ownership table for the team.
179. Check that every required route works when entered directly in the browser.
180. Demonstrate one successful and one unsuccessful authentication scenario.
181. Show one Firestore read and one write.
182. Prepare each member's branch, pull request and commit evidence.
183. Rehearse the individual viva: feature explanation, code walkthrough, debugging story and one live modification.

Verification Checklist

- At least three routes are available.
- Authentication and protected routes work.
- At least one Firestore collection supports basic read/write.
- At least one meaningful ShadCN/ui feature is used.
- Responsive design and deployment are functional.
- README contains setup, features and contribution information.
- Each member can demonstrate their end-to-end feature and individual evidence.

Lab Record and Reflection

Record item	Student notes
What I implemented	
One issue or error I encountered	
How I investigated and fixed it	
How I verified the result	
What I can now explain or modify independently	

AI-ASSISTED LAB ACTIVITY

AI Activity 15 - AI-Assisted Release Readiness Review

Copyable prompt - replace bracketed text with your own work:

> Act as a release-readiness reviewer for my student group project. Based on the requirements and evidence I provide, create a checklist for routes, authentication, Firestore basic read/write, responsive behavior, accessibility basics, Lighthouse findings, README, deployment and GitHub contribution evidence. Separate confirmed evidence from missing evidence. Do not assume a feature works just because it is described.

Procedure

1. Provide AI with the project requirements and a list of actual test results, not just feature claims.
2. Ask it to produce a gap checklist. Mark each item confirmed, failed or not tested based on evidence.
3. Run the missing tests: routes, authentication, Firestore, responsive checks and the relevant Lighthouse audit.
4. Ask AI to review your README for missing setup steps and unclear contribution notes. Verify every command and URL.
5. Use AI to generate likely viva questions for your own feature, then answer and demonstrate them without AI.
6. Do a final human review of every pull request and confirm each member's own branch, PR and eight meaningful commits.

Required evidence / submission

• Release-readiness checklist showing confirmed, failed and not-tested items.

• Evidence for the tests performed and the remaining issues resolved or documented.

• Each member's feature, branch, pull request and commit evidence.

Verification rule: run the code, test the behavior and check important claims against browser evidence or official documentation. You remain responsible for understanding and modifying the submitted work.

Mid-term Practical Examination Guide

Item	Specification
Scheduled block	Week 7 C3–C4
Duration	100 minutes
Marks	20
Nature	Individual practical assessment
Permitted scope	Semantic HTML, CSS, responsive layout, JavaScript fundamentals, DOM and meaningful interaction covered before the mid-term.
Not in stated scope	React and Firebase.
Assessment emphasis	Implementation, problem-solving, responsiveness, interaction, testing, debugging and individual understanding.

Suggested Examination Workflow

184. Read the full problem statement and identify inputs, outputs and constraints.
185. Sketch the page structure and identify the interaction required.
186. Build semantic HTML before styling.
187. Apply CSS and check the layout at the expected viewport sizes.
188. Implement JavaScript interaction and validation.
189. Test normal, invalid and boundary inputs.
190. Use DevTools to investigate errors and verify the final behavior.
191. Review the page and be prepared to explain your decisions.

The instructor specifies permitted AI-use conditions before the examination. Do not use AI to bypass individual demonstration or examination rules.

Final Group Project - Student Specification

Teams of three are formed in Week 8. Teams may propose their own problem and application idea, subject to instructor approval. The final project integrates React, routing, ShadCN/ui, Firebase Authentication, Firestore and deployment.

No.	Minimum requirement
1	At least three routes.
2	Authentication with protected routes.
3	At least one Firestore collection with basic read/write.
4	At least one meaningful ShadCN/ui feature.
5	Responsive design and deployment.
6	README with setup, features and contribution information.
7	Each member owns at least one feature end-to-end.
8	Each member works on an individual branch and opens a pull request.
9	Each member makes at least eight meaningful commits.
10	Each member completes an individual viva and demonstrates their own contribution.

Project Planning Worksheet

Planning question	Team response
What problem are we solving?	
Who are the intended users?	
What are the three or more routes?	
What features are essential for the first working version?	
What data will the Firestore collection store?	
Which authentication flow will be implemented?	
Which ShadCN/ui feature will be used and why?	
How will work be divided among three members?	
What are the first backlog items and their priorities?	

Feature Ownership Matrix

Member	Owned feature	Branch	Pull request	Commit evidence
Member 1				
Member 2				
Member 3				

Suggested Viva Questions

- Which feature did you implement end-to-end?
- Show the route or component that implements it.
- How does data move through your feature?
- What state or authentication condition controls its behavior?
- Show the branch and pull request containing your changes.
- Describe one defect you encountered and how you verified the fix.
- Make a small change to your feature and explain the result.

AI-Assisted Development Policy and Log

No.	Course policy
1	AI tools are permitted during regular laboratory development activities; students may choose their preferred assistant.
2	Do not blindly accept generated code. Run, test and verify each implementation.
3	You are responsible for understanding, explaining, modifying and debugging all submitted work.
4	Acknowledge significant AI contributions in the project README or development log.
5	Do not submit credentials, private keys, personal data or confidential information to AI tools.
6	Review AI output for correctness, accessibility, security, maintainability and suitability for the task.
7	AI use does not replace individual assessment, live demonstration or viva.

Recommended Prompting Pattern

A useful request to an AI assistant includes: the task context, existing code or relevant structure, expected behavior, constraints, the error or uncertainty, and the form of help you want. Ask for an explanation when you do not understand a suggestion.

Development log field	Record
Project / date	
Problem understanding	
AI tool(s) used	
Representative prompt(s)	
What AI suggested	
What I accepted, changed or rejected	
Implementation decisions and reasons	
Tests performed and results	
One error investigated and resolution	
What I can now do independently	

AI Verification Checklist

- Does the code solve the actual requirement?
- Can I explain every important function, component or configuration?
- Have I run the code rather than assuming it works?
- Have I tested invalid inputs, empty data and failure cases where relevant?
- Does it introduce accessibility, security or maintainability problems?
- Have I acknowledged significant AI assistance?

Quick Reference and Common Troubleshooting

Problem	What to inspect first
HTML page is blank or incomplete	File path, document structure, browser console and whether the correct file is open.
CSS changes do not appear	Stylesheet link, filename/path, selector match, specificity and cached resources.
Layout overflows on mobile	Fixed widths, large images, long text, grid/flex minimum sizes and media query behavior.
JavaScript event does not run	Script loading, selector result, event name, element timing and console errors.
DOM element is null	Check selector spelling and whether the script runs after the element exists.
Fetch request fails	Request URL, Network tab, status code, connectivity, CORS message and response format.
localStorage data cannot be parsed	Stored value, JSON format, parse errors and fallback behavior.
React component does not update	State mutation, setter usage, prop values, keys and component console warnings.
React route shows 404 unexpectedly	Route path, router setup, nested route layout and direct-refresh hosting behavior.
Firebase read/write fails	Project configuration, enabled service/provider, auth state, collection path and supplied security rules.
Git push or pull request issue	Current branch, remote URL, commit status, permissions and whether the branch is pushed.

Core Documentation

Resource	Use
MDN Web Docs	HTML, CSS, JavaScript, DOM and browser APIs.
web.dev - Learn CSS / Learn Forms	CSS, responsive design and forms.
javascript.info	JavaScript and browser-side programming.
Eloquent JavaScript	Supplementary JavaScript reference.
React Documentation	Components, state, effects and React patterns.
Vite Documentation	Development environment and project setup.
React Router Documentation	Routing and navigation.
Tailwind CSS Documentation	Utility-first responsive styling.
ShadCN/ui Documentation	Practical component usage.
Firebase Documentation	Authentication and Firestore.
Git / GitHub Documentation; Pro Git, 2nd Edition	Version control, branches and pull requests.
Figma / Pixso Documentation	Design handoff and asset inspection.

Weekly AI Activity Evidence Rubric

This is a formative guide for evaluating AI-assisted learning; it does not change the official assessment marks in the syllabus.

Criterion	Satisfactory evidence	Suggested emphasis
Prompt quality	Task, context, constraints and requested assistance are clear.	20%
Technical verification	Output is checked through execution, tests, Developer Tools or official documentation.	30%
Student ownership	Student can explain, modify and debug the implementation independently.	30%
Reflection and integrity	Accepted/rejected suggestions are recorded; private information is protected.	20%

Final Submission Checklist

- The application runs from the documented setup instructions.
- Required features work and have been tested.
- Responsive behavior and relevant accessibility basics have been checked.
- The repository and deployed URL are correct and accessible.
- The README contains the required setup, feature and contribution information.
- Significant AI assistance is acknowledged and the development log is complete.
- For team work, each member can identify their own feature, branch, pull request and at least eight meaningful commits.
- Each member can explain and modify their own implementation during viva.

Scope Reminder

This manual follows the revised syllabus's stated scope. The Week 14 Firebase work is specified without Context API. Firestore is limited to one collection with basic read/write and the supplied security-rules starter. The React UI library work is the practical subset named in the syllabus. Do not assume that advanced topics outside the stated scope are required for assessment.